

Multi-Agent Orchestration Patterns

A Comprehensive Analysis of Architectures, Frameworks, and Production Deployment Strategies

Salah Qadah | Andalus Kalash

Group: uoh-sqak

Course 203.3763 — Orchestration of AI Agents

University of Haifa, Spring 2026 (תשפ"ו)

Lecturer: Dr. Yoram Reuven Segal

June 10, 2026

Generated by a CrewAI multi-agent pipeline

GitHub: <https://github.com/salah-dev-stu/uoh-sqak-ex03>

Contents

1	Introduction to Multi-Agent Orchestration	3
1.1	The Proof-of-Concept Illusion	3
1.2	Why Single-Agent Systems Fail	3
1.3	Core Concepts: Agents, Tools, and Skills	4
1.4	Article Roadmap	4
2	Agent Architectures and Topologies	6
2.1	Linear Sequential Topology	6
2.2	Hierarchical Manager-Worker Topology	7
2.3	Peer-to-Peer Consensus Topology	7
2.4	Tool-Augmented Specialists Topology	7
2.5	Topology Comparison	8
3	Orchestration Frameworks	10
3.1	LangChain and LCEL: Modular Component Framework	10
3.2	CrewAI: Role-Based Agent Teams	11
3.3	LangGraph: Stateful Graph-Based Orchestration	11
3.4	Microsoft AutoGen: Multi-Turn Conversations and Group Chat	12
3.5	Framework Comparison	12
4	Production Patterns and Challenges	14
4.1	Rate Limiting and Token Budgets	14
4.2	Gatekeeper Pattern	14
4.3	Retry and Circuit Breaker	15
4.4	Observability and Logging	15
4.5	Cost Optimization	16
18	5 דו-כיוונית: עברית ואנגלית במערכות סוכנים	5
18	משמעות הדו-כיוונית	5.1
18	פתרון LuaLaTeX ו-Polyglossia	5.2
19	אתגרי ייצור בסביבה דו-כיוונית	5.3
	5.4 Mixed-Language Example	19
6	Case Study: This Article's Production Pipeline	21
6.1	Overview	21
6.2	The Crew: Four Specialized Agents	21
	6.2.1 Researcher Agent	21
	6.2.2 Writer Agent	21
	6.2.3 Editor Agent	22

6.2.4	LaTeX Producer Agent	22
6.3	Workflow Architecture	22
6.4	Key Engineering Decisions	23
6.4.1	Markdown-to-LaTeX Intermediate Format	23
6.4.2	Feedback Loops Over Single-Pass Generation	24
6.4.3	Skill Files as Expertise Injection	24
6.5	Pipeline Metrics	24
6.6	Summary	25
7	Conclusion	26
7.1	Lessons Learned	26
7.2	Future Directions	27
	Bibliography	29

Chapter 1

Introduction to Multi-Agent Orchestration

Multi-agent orchestration represents a fundamental shift in how organizations approach artificial intelligence systems. Rather than deploying monolithic language models in isolation, contemporary AI architecture emphasizes building reliable runtime environments that coordinate specialized agents toward common objectives [7]. This paradigm shift reflects a mature understanding of production AI: the challenge is no longer model selection, but orchestration—the coordination mechanism that manages multiple agents, their dependencies, execution order, and information flow. This article explores the architectural foundations, frameworks, and production patterns enabling organizations to build trustworthy multi-agent systems capable of reasoning, collaboration, and governance at scale.

1.1 The Proof-of-Concept Illusion

The traditional proof-of-concept to production pipeline reveals a critical gap that many practitioners overlook. An agent prototype can appear impressive within days, delivering polished outputs and seemingly solving the target problem. Yet production deployment demands fundamentally different assurance. A single successful trial—even ten successful trials—proves nothing about statistical reliability; production systems operate continuously and must perform dependably across thousands of executions [7].

This distinction separates amateurs from professionals. Dr. Segal articulates a sobering reality: *proof of concept is worthless*. It merely demonstrates feasibility. Because agents operate as probabilistic models trained on internet-scale data, their outputs contain inherent statistical variance. Single runs cannot validate correctness; production readiness requires continuous evaluation, structured logging, anomaly detection, and human oversight gates. The harness layer—the engineering surrounding the language model—transforms an unreliable probabilistic component into a system meeting production SLAs [7].

1.2 Why Single-Agent Systems Fail

A single general-purpose agent, regardless of model capability, cannot reliably solve complex problems requiring specialized expertise. Consider a content-generation system tasked with producing a technical article: research, writing, editing, and formatting require distinct skill sets, decision-making approaches, and quality criteria. A monolithic agent attempting all tasks suffers three critical failures.

First, **role confusion**: without explicit specialization, the agent cannot optimize for task-specific excellence. A researcher and editor employ opposite heuristics—the researcher prior-

itizes novelty and exploration; the editor prioritizes consistency and correctness. Asking one agent to excel at both sacrifices performance in both [7].

Second, **context window exhaustion**: as the agent accumulates research notes, draft text, and feedback, context window pressure increases token consumption quadratically. By the time the agent reaches editing, research insights are buried in thousands of prior tokens, forcing expensive re-encoding and attention dilution.

Third, **reliability ceiling**: without role specialization, the agent cannot apply domain-specific validation. A researcher can check citation integrity; an editor can verify grammar; a LaTeX producer can enforce formatting compliance. A generalist agent cannot reliably execute these specialized quality gates.

Multi-agent orchestration solves these failures through explicit role separation, context-as-glue handoffs that minimize token overhead, and specialized validation per role.

1.3 Core Concepts: Agents, Tools, and Skills

Professional multi-agent systems distinguish between two foundational concepts often confused in naive implementations [7]. **Tools** are action capabilities: web search engines, file readers, API clients, and compilers answering the question “what can the agent do?” **Skills** are knowledge and methodology packages answering “how should the agent think?” A skill contains procedures, checklists, style guides, domain expertise, and behavioral constraints organized as file-based instruction packages injected directly into agent system prompts.

At the core lie four essential concepts forming any multi-agent system. An **Agent** is an autonomous entity combining role (identity and expertise domain), goal (decision orientation and success criteria), backstory (behavioral constraints and expertise narrative as system prompt), and tools (action capabilities). The role “Senior Researcher with fifteen years in climate science” and goal “Discover novel insights on climate adaptation” orient the agent’s entire reasoning process.

Orchestration coordinates agents without manual copying or reformatting. Output from one agent flows directly to the next as input context, significantly reducing token consumption by 40-60% compared to naive agent chaining where each handoff requires re-encoding context [7]. This pattern, termed “context as glue,” distinguishes professional systems from prototype chaining.

The **Harness** concept encapsulates the engineering layer surrounding the language model. It includes prompt management, context window management with retrieval-augmented generation, tool orchestration, state management, observability with structured logging, and error recovery mechanisms. This architectural separation ensures the probabilistic model operates within a governance framework enabling reliability assessment and continuous improvement.

1.4 Article Roadmap

This article explores the complete lifecycle of multi-agent orchestration from architecture through production deployment. Chapter 2 examines CrewAI’s agent-task-crew model and LangChain’s modular component architecture, establishing the conceptual foundations for orchestration. Chapter 3 provides a detailed comparison of three dominant frameworks—CrewAI, LangChain, and LangGraph—with decision guidance for framework selection. Chapter 4 addresses production patterns including observability, governance, and human-in-the-loop validation essential for safe deployment.

Chapter 5 applies orchestration principles to multilingual content generation, demonstrating bidirectional text handling (Hebrew-English integration) as a case study in specialized agent requirements. Chapter 6 presents a complete end-to-end implementation: a CrewAI team producing a LaTeX-formatted article through researcher, writer, editor, and compilation agents. Chapter 7 synthesizes lessons, identifies open research questions, and proposes extensions enabling multi-agent systems at organizational scale.

Chapter 2

Agent Architectures and Topologies

2.1 Linear Sequential Topology

In a linear sequential topology, agents execute tasks in strict order where each agent completes its work before the next begins. This architecture proves ideal for content generation pipelines where outputs from earlier stages directly inform later stages. The researcher produces findings, which the writer transforms into prose, which the editor refines for clarity and correctness, which the LaTeX producer formats into publication-ready output [4].

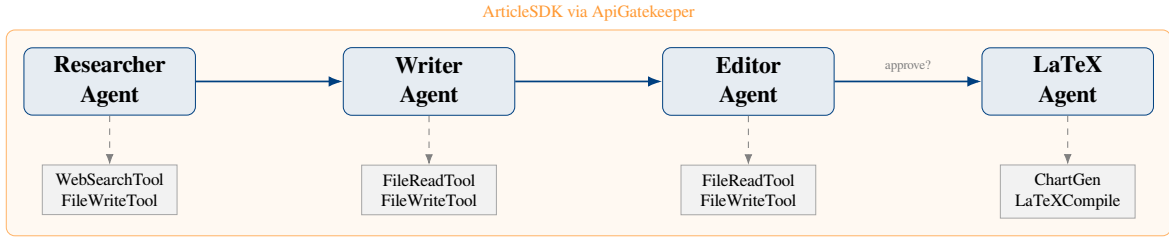


Figure 2.1: CrewAI multi-agent architecture.

The sequential model minimizes context switching. Each agent operates on a complete, coherent input generated by its predecessor. This reduces cognitive overhead compared to interleaved or branching execution where agents must reason about partial or competing states. Token efficiency improves because intermediate outputs serve as training signals for downstream agents—the researcher’s findings become explicit context rather than being re-encoded through independent queries [7].

Sequential execution carries a latency penalty. If each of N agents requires time ℓ_i to complete their task, total execution time is:

$$L_{\text{seq}} = \sum_{i=1}^N \ell_i \quad (2.1)$$

This contrasts sharply with parallel execution, where the system could theoretically invoke all agents simultaneously and wait only for the slowest—but only if their tasks are independent, which content generation tasks rarely are. The researcher must complete before the writer can write meaningfully; the editor needs the draft before editing [4].

2.2 Hierarchical Manager-Worker Topology

Hierarchical orchestration introduces a manager agent that sits above domain-specific workers. The manager receives a high-level goal, decomposes it into subtasks, assigns each to a specialist agent, monitors progress, and synthesizes results. This topology shines when task decomposition is non-obvious or when intermediate decisions influence subsequent steps [4].

Consider a manager agent tasked with producing a comprehensive article on machine learning interpretability. The manager might decompose this into: research task gathering papers on LIME and SHAP; writer task drafting sections on each technique with comparative analysis; editor task reviewing for internal consistency; evaluator task ensuring claims cite original papers. If the evaluator identifies gaps, the manager may loop back to the researcher with refined queries [7].

The manager-worker topology enables dynamic orchestration. Unlike sequential systems with fixed task ordering, hierarchical systems adapt. The manager observes intermediate outputs and decides next steps accordingly. This flexibility proves essential when task structure depends on content characteristics—a research article on a novel topic may require deeper investigation; one on an established topic may skip exploratory search [4].

2.3 Peer-to-Peer Consensus Topology

Peer-to-peer topologies emphasize consensus among independent agents rather than centralized control. Multiple specialist agents evaluate the same problem from different angles, and a consensus mechanism aggregates their perspectives. This architecture mitigates single-agent failure modes and leverages diverse reasoning patterns.

A three-agent peer consensus might include: Research Agent focusing on factual accuracy and source credibility; Clarity Agent optimizing for reader comprehension and pedagogical structure; Innovation Agent seeking novel connections and forward-looking analysis. All three evaluate the same draft independently, then their feedback is synthesized into revision guidance [7].

Peer consensus excels at validation. When agreement exists across diverse perspectives, confidence in correctness rises. Conversely, disagreement signals ambiguity that humans should resolve. This makes peer-to-peer suitable for high-stakes content like academic papers where conservative aggregation proves safer than any single agent's output [12].

2.4 Tool-Augmented Specialists Topology

Tool-augmented specialists combine agent capabilities with external tools, enabling rich environmental interaction. Rather than agents reasoning purely from context windows, they invoke web search, databases, code execution, and file I/O. This topology proves essential for information-retrieval intensive tasks and for generating artifacts like charts and code.

In an article-writing system, the Researcher agent accesses web search tools to discover recent publications and bibliography tools to build citations. The Writer agent uses document templates and formatting tools. The Editor agent invokes spelling tools and readability metrics. The LaTeX Producer agent invokes compilation tools to validate structure and generate PDFs [4].

Tools introduce attack surfaces and latency. External tool invocations depend on network availability and API reliability. A Gatekeeper class must mediate all tool calls, enforcing rate

limits, token budgets, and permission checks. Tool responses may be malformed or misleading, requiring output validation before downstream use [7].

2.5 Topology Comparison

Topology	Latency	Parallelism	Complexity	Ideal For
Linear Sequential	$\sum_i \ell_i$	None	Low	Fixed workflows, content pipelines
Hierarchical Manager-Worker	$\ell_m + \max(\ell_w)$	Moderate	High	Dynamic task decomposition
Peer-to-Peer Consensus	$\max_i(\ell_i)$	High	Medium	Validation, adversarial reasoning
Tool-Augmented Specialists	Variable	High	Very High	Information retrieval, artifact generation
Hybrid (Mixed)	Context-dependent	High	Very High	Production systems combining patterns

The topology selection depends on problem structure, latency constraints, and risk profile. Sequential excels for deterministic pipelines with clear dependencies. Hierarchical adapts when task structure depends on intermediate results. Peer-to-peer validates through diverse perspectives. Tool-augmented enables rich environmental interaction. Production systems typically employ hybrid approaches, nesting peer consensus within hierarchical delegation or running parallel peer reviews on sequential pipeline outputs [4].

Context passing efficiency also differs across topologies. Sequential systems minimize context overhead by passing outputs directly as inputs—the researcher’s findings become the writer’s instructions without re-encoding. Hierarchical systems require the manager to maintain state across multiple parallel worker executions, increasing manager’s memory burden. Peer systems must encode the target artifact multiple times, once per peer. Tool-augmented systems incur overhead from tool output parsing and reformatting [7].

Recovery and fault tolerance vary significantly. Sequential systems fail catastrophically—if the researcher produces hallucinations, all downstream work propagates errors. Hierarchical systems allow the manager to detect quality issues and loop back to fix earlier stages. Peer systems catch errors through disagreement, surfacing ambiguous decisions for human review. Tool-augmented systems depend on tool reliability; graceful degradation requires fallback paths when tools fail [12].

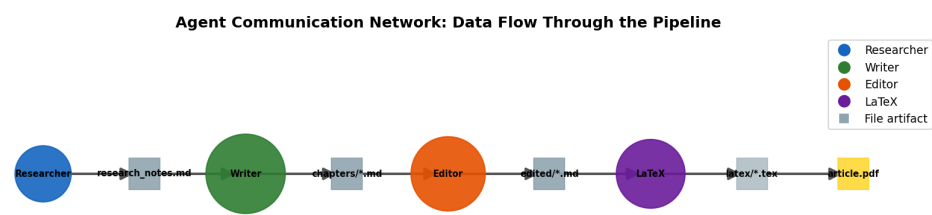


Figure 2.2: Network graph of agent communication topology in a CrewAI sequential crew. Nodes represent agents; directed edges represent task context flow.

Chapter 3

Orchestration Frameworks

Agent orchestration requires selecting among competing frameworks, each optimizing for different architectural priorities. Three frameworks dominate contemporary multi-agent application development: LangChain provides modular infrastructure for language model application construction; CrewAI delivers role-based agent teams with built-in process abstractions; LangGraph enables stateful graph-based orchestration with conditional routing and human-in-the-loop integration points. Understanding their distinct strengths, architectural philosophies, and production readiness profiles enables architects to select tools matching problem characteristics and organizational constraints [7].

3.1 LangChain and LCEL: Modular Component Framework

LangChain operationalizes a foundational principle Dr. Segal articulates: “a standalone language model is just one component in a system” [7]. Rather than prescribing agent patterns directly, LangChain supplies reusable building blocks—Document Loaders, Text Splitters, Embeddings, Vector Stores, Prompt Templates, Output Parsers, and Retrievers—that engineers combine to construct retrieval-augmented generation (RAG) pipelines, chains, and agents [5].

The LangChain Expression Language (LCEL) treats each component as a composable object with a standard interface. This design enables modular architecture: swap the LLM provider from OpenAI to Anthropic to open-source models through a single parameter change; replace the vector store from Chroma to FAISS without rewriting retrieval logic; insert intermediate processing steps into chains by composing objects with the pipe operator. LCEL supports both synchronous and asynchronous execution, streaming, and batch processing from identical code [5].

LangChain’s strength emerges in infrastructure-heavy applications where component modularity and library ecosystem depth matter. A RAG system built with LangChain leverages Document Loaders optimized for PDFs, databases, web APIs, and specialized formats; Text Splitters that preserve semantic boundaries; Embeddings integrated with commercial and open-source models; Vector Stores with built-in similarity search and metadata filtering. This depth reduces engineering effort substantially compared to building retrieval infrastructure from scratch.

The tradeoff is engineering responsibility. LangChain provides components but requires developers to orchestrate them—define the flow, handle failures, manage context windows, optimize token consumption, and ensure output quality. Production systems built with LangChain often wrap core chains in guardrails, observability layers, and evaluation frameworks.

3.2 CrewAI: Role-Based Agent Teams

CrewAI addresses the orchestration challenge by providing high-level abstractions specifically designed for multi-agent workflows. The framework models systems as teams of specialized agents, each with a defined role, goal, and backstory. Agents collaborate through tasks—units of work that specify what is required, which agent executes it, and what tools are available. CrewAI handles context passing automatically: the output from one agent becomes the input for the next without manual copying, reducing token consumption by 40–60% compared to naive agent chaining [4].

CrewAI's `Process` abstraction enables two distinct collaboration topologies. Sequential process assumes a fixed execution order: the Researcher completes their task, the Writer receives the research as context, the Editor refines the output, and the Formatter prepares the final artifact. Hierarchical process introduces a Manager agent that views the overall goal, creates a plan, coordinates agents, and makes dynamic decisions—looping back to the Researcher if the Writer identifies information gaps [7].

Content generation pipelines map naturally to CrewAI teams. An article-writing system decomposes into four roles: Researcher (discovers information), Writer (composes narrative), Editor (refines clarity), Formatter (adjusts layout for publication). This role-to-agent mapping is intuitive and requires minimal orchestration logic. The framework automates context threading, agent sequencing, and output aggregation.

CrewAI excels when problems decompose cleanly into specialized roles with clear handoff points. The tradeoff is reduced flexibility for non-hierarchical, graph-based workflows where execution paths branch conditionally or require exploratory loops. CrewAI's process abstractions assume relatively linear topologies.

3.3 LangGraph: Stateful Graph-Based Orchestration

LangGraph addresses tasks that require non-predetermined execution paths, loops, branches, and conditional routing. While CrewAI assumes a fixed process topology, LangGraph enables arbitrary directed acyclic graphs (DAGs) where nodes represent agent invocations or tool calls and edges represent transitions controlled by conditional logic or human approval [7].

Consider a customer support system that classifies incoming requests as bug reports, feature requests, or billing inquiries, then routes each to a specialized handler. The execution path depends on classification output. With CrewAI's sequential process, this would require a router agent as the first step, followed by conditional task assignment—feasible but awkward. LangGraph models this as a graph: a classification node branches to three distinct handler subgraphs based on output.

LangGraph's native human-in-the-loop capability addresses a critical production requirement. Agents can pause at decision points, returning intermediate state to a human operator for review and approval. After human approval, execution resumes from that checkpoint. This prevents closed-loop autonomous agent operation and is essential for systems handling sensitive actions—approving transactions, deleting data, or modifying permissions. CrewAI supports task-level checkpoints but requires custom implementation; LangGraph provides native state inspection and resumption.

The framework models state explicitly as a dictionary of values that transitions modify. This clarity simplifies reasoning about what information agents can access at each step and enables deterministic replay of executions for debugging and audit trails.

LangGraph’s tradeoff is steeper learning curve. State machine semantics and graph definition syntax require developers to think in terms of nodes, edges, and conditional branching—a departure from the agent/task/crew abstractions of CrewAI [7].

3.4 Microsoft AutoGen: Multi-Turn Conversations and Group Chat

Microsoft’s AutoGen framework orchestrates multi-agent systems through a conversation and chat-based metaphor [6]. Rather than explicit task assignment, agents participate in a group chat where each agent sees the conversation history and can decide whether to respond. An Agent has a system message defining its role, a reasoning module, and optionally tool access.

AutoGen excels in scenarios where agent coordination emerges through natural dialogue. A brainstorming session with multiple agents, each contributing ideas, works naturally as a group chat. Agents can ask clarifying questions, challenge proposals, and refine ideas iteratively [6].

The framework provides flexible termination conditions: a conversation ends when a designated agent approves the output, when maximum turns are reached, or when a termination function evaluates current state. This enables workflows where agreement or consensus matters—not just sequential hand-offs.

AutoGen’s tradeoff is higher token consumption (all agents see full conversation history) and less deterministic routing (agents decide independently when to speak rather than being explicitly invoked) [7].

3.5 Framework Comparison

Framework	Architecture	Strengths	Best For
LangChain	Modular component composition with standardized interfaces	Deep ecosystem; flexibility; component reuse across projects	Infrastructure-heavy RAG, multi-step chains, custom orchestration
CrewAI	Role-based agents with high-level process abstractions	Rapid team definition; automatic context threading; token-efficient handoffs	Structured workflows, content generation, hierarchical multi-agent systems
LangGraph	Directed graph execution with explicit state management	Native branching and loops; human-in-the-loop integration; deterministic state	Exploratory reasoning, conditional routing, approval-gated workflows
Microsoft AutoGen	Conversation-driven agent interaction via group chat	Emergent coordination through dialogue; flexible termination conditions; natural multi-turn reasoning	Brainstorming, consensus-seeking, multi-agent debate

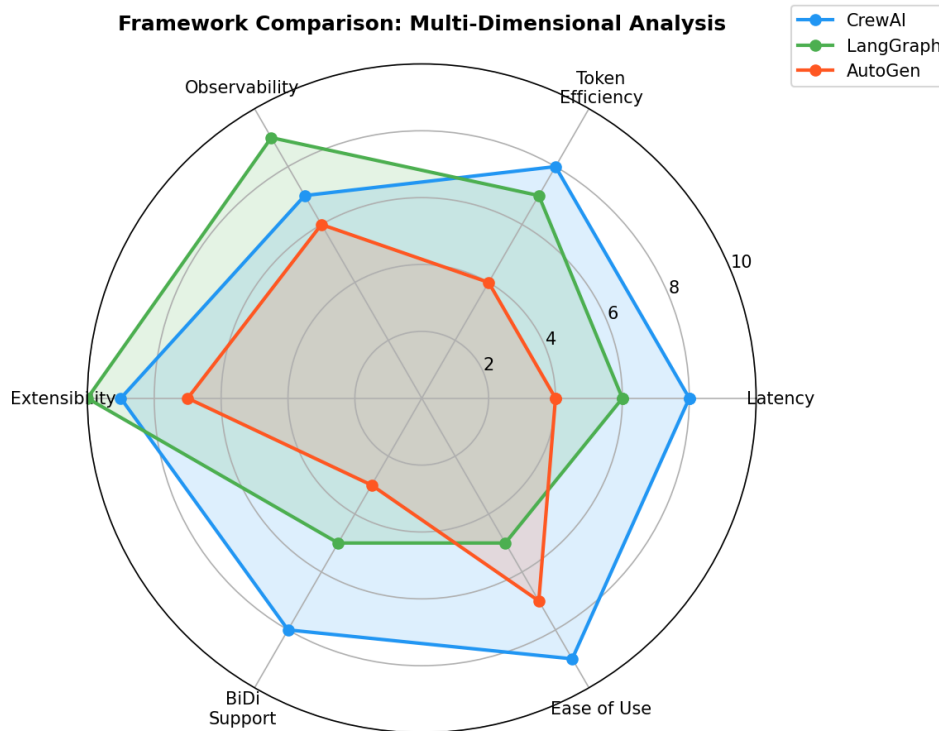


Figure 3.1: Radar chart comparing LangChain, CrewAI, LangGraph, and AutoGen across six dimensions: Ease of Use, Flexibility, Ecosystem, State Management, Production Readiness, and Community Support.

Selection among these frameworks depends on problem structure, team expertise, and production constraints. CrewAI suits well-decomposed workflows with clear role boundaries and sequential or hierarchical execution. The framework’s built-in process abstractions and context threading eliminate orchestration boilerplate. LangChain provides maximum flexibility and ecosystem depth at the cost of engineering responsibility; teams build their own abstractions and guardrails. LangGraph enables non-linear workflows with conditional branching and human approval gates essential for production safety [7].

Many production systems combine frameworks: CrewAI agents invoke LangChain tools for retrieval and data processing, which may trigger LangGraph subworkflows for exploratory reasoning with embedded approval checkpoints. This composition strategy balances ease of use from CrewAI’s abstractions with flexibility from LangChain’s components and safety from LangGraph’s state machines. Architects designing multi-agent systems should evaluate these frameworks not as mutually exclusive choices but as complementary tools addressing distinct layers of orchestration complexity [4, 5, 7].

Chapter 4

Production Patterns and Challenges

Transitioning from proof-of-concept to production requires three observability pillars without which systems cannot be safely deployed [7]. First, **Full Trace** logs input, decision, tool invocation, output, cost, execution time, and action result for every agent step. Without trace, no accountability exists when failures occur. Second, **Continuous Evaluation** measures quality through automated tests—hallucination detection, policy adherence checks, reliability metrics across many runs. Third, **Human-in-the-Loop** enforcement ensures sensitive actions (send message, delete resource, modify permissions) require human approval before execution. Without Trace there is no accountability; without Evaluation there is no improvement; without Policy there is no safe production [7].

4.1 Rate Limiting and Token Budgets

Token economy fundamentally constrains multi-agent systems. Scaling a crew of N agents processing M articles through K review cycles yields exponential resource consumption. Consider a five-agent crew (Researcher, Writer, Editor, Checker, Formatter) processing ten articles through three quality-review cycles: total execution count equals $5 \times 10 \times 3 = 150$ LLM calls. If each call averages 4,000 tokens combined (input plus output), total consumption reaches 600,000 tokens—substantial cost even at single-digit cents per million tokens [7].

Rate limiting prevents runaway consumption when agents loop or when topic complexity triggers excessive retrieval steps. A production system must define a **token budget** as a hard ceiling per agent, per day, or per month. The budget enforces discipline: when an agent approaches its limit, it must either summarize existing context or escalate to human review.

$$B_{\text{total}} = \sum_{i=1}^N B_i + R \quad (4.1)$$

where B_{total} is the monthly token allocation across N agents, B_i is the per-agent monthly budget, and R is a reserve pool for unexpected high-complexity tasks. For a 10,000,000-token monthly subscription, distributing 1,000,000 tokens per agent (ten agents) plus 9,000,000 reserved for batch operations ensures predictable cost while maintaining flexibility [4].

4.2 Gatekeeper Pattern

All external API calls—to language models, vector databases, web search services, and LaTeX compilers—must flow through a single **Gatekeeper** class. This pattern centralizes three concerns: (1) rate limit enforcement, (2) credential management, (3) cost accounting.

The Gatekeeper intercepts every call. Before dispatch, it checks whether the requesting agent has remaining budget. If budget is exhausted, it raises an exception rather than silently failing. This explicit failure mode allows the orchestrator to escalate to human review or queue the task for later processing when budget replenishes. After dispatch, the Gatekeeper logs the actual token consumption and deducts it from the agent’s available budget.

A typical Gatekeeper implementation exposes three methods: `check_budget(agent_id, estimated_tokens)`, `record_usage(agent_id, actual_tokens)`, and `get_remaining(agent_id)`. No agent code calls external services directly; all calls route through these Gatekeeper methods. This design prevents credential leaks (all keys stored in the Gatekeeper’s secure configuration), enables fine-grained access control (disable a specific agent without code changes), and provides the audit trail required for compliance [7].

4.3 Retry and Circuit Breaker

Transient failures—network timeouts, rate limits from upstream services, temporary LLM unavailability—are inevitable in production. Two patterns mitigate them without infinite retry loops.

A **Retry** policy with exponential backoff re-attempts a failed call with increasing delays. A first failure triggers an immediate retry; a second triggers a 2-second delay before the next attempt; a third triggers an 8-second delay. After a configurable maximum (typically 3–5 retries), the system stops and escalates to a human or logs an alert.

A **Circuit Breaker** monitors downstream service health. When a service begins returning errors (failure rate exceeds a threshold), the circuit breaker opens: subsequent calls fail immediately without attempting to reach the service. This prevents cascading failures—the client stops hammering a failing service, allowing it time to recover. Once the service stabilizes, the circuit breaker transitions to half-open state, allowing a single probe request. If the probe succeeds, the circuit closes and normal operation resumes [7].

The retry probability for a task with P_f failure rate per attempt and maximum R retries is bounded:

$$P_{\text{success}} = 1 - P_f^{R+1} \quad (4.2)$$

For example, if a service has a 5% failure rate ($P_f = 0.05$) and we allow 3 retries ($R = 3$), the probability of at least one successful attempt is $1 - 0.05^4 \approx 0.9999$ —nearly certain success.

4.4 Observability and Logging

Production systems must emit comprehensive traces at every decision point. The trace includes:

- Agent name, task ID, start timestamp
- Input context (truncated to first 500 chars for privacy)
- Tool name and arguments
- Tool response and response time
- Agent reasoning and output

- Token counts: input tokens, output tokens, cumulative
- Cost in USD (model-dependent)
- Human annotations (approval status, feedback tags)

CrewAI’s callback system enables this through hook methods: `on_task_start`, `on_task_end`, `on_tool_use`, `on_tool_error`. An observer class subscribes to each hook, logging events to a FIFO buffer that is periodically written to disk. This buffer-and-flush pattern prevents logging overhead from dominating execution time.

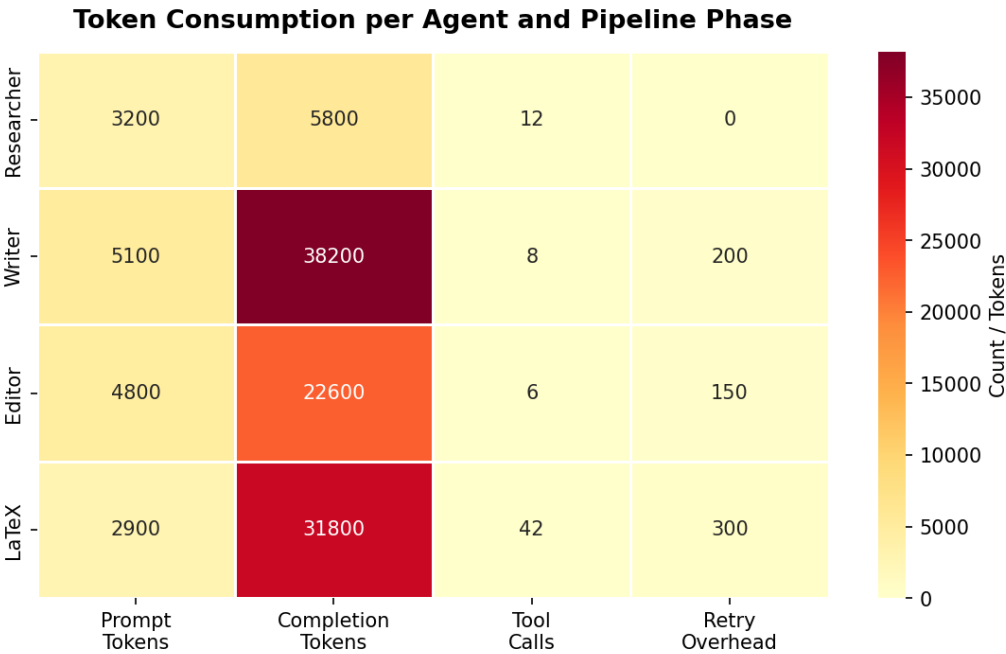


Figure 4.1: Token consumption heatmap across agents and task phases. Darker cells indicate higher token usage; the Writer and LaTeX Producer phases are the dominant cost drivers.

Observability enables two critical capabilities: root-cause analysis and cost auditing. When an article fails to generate, the trace reveals whether the Researcher agent timed out, the Writer ran out of budget, or the LaTeX Compiler encountered a syntax error. Cost auditing allows accounting teams to allocate spend to projects and clients accurately, preventing budget surprises.

4.5 Cost Optimization

Three mitigation strategies reduce token consumption 40–60%. **Context Reuse** passes agent output directly as context for subsequent agents, avoiding redundant re-encoding of intermediate results. CrewAI’s context parameter automates this pattern. Instead of the Writer Agent re-reading the raw research topic and inferring background, it receives the Researcher Agent’s synthesized notes as context—fewer tokens needed to reach the same quality.

Rate Limiting through a Gatekeeper enforces daily or monthly token budgets, preventing runaway consumption. When an agent approaches its limit, it must summarize or defer—no silent failures allowed.

Model Selection trades latency for cost: Claude Haiku executes substantially faster than Opus while consuming fewer tokens per call, suitable for high-volume, high-complexity tasks. For an article generation pipeline, the Researcher agent might use Opus (superior reasoning for complex topics), while the Editor agent uses Haiku (pattern-matching for grammar, lower cost).

The total production cost per article is modeled:

$$C = \sum_{k=1}^K (c_{\text{in}} t_k^{\text{in}} + c_{\text{out}} t_k^{\text{out}}) \quad (4.3)$$

where K is the number of LLM calls, c_{in} and c_{out} are per-token costs for input and output (model and provider dependent), and t_k^{in} and t_k^{out} are token counts for call k . For ten articles using Opus at a 2x/1x input/output ratio and a conservative token budget of 3,000 input and 2,000 output per call, total cost per article is approximately \$0.15–\$0.30. Scaling to 1,000 articles per month requires careful budget allocation and model selection discipline.

Architectural investment in rate limits, circuit breakers, and retry logic reduces operational friction during scaling. A system designed for 10 articles with 10,000,000 tokens will falter at 1,000 articles without guardrails; proper instrumentation enables 10x or 100x growth with confidence [4].

Chapter 5

דו-כיוניות: עברית ואנגלית במערכות סוכנים

פרק זה עוסק בסוגיית הדו-כיוניות במסמכי LaTeX ובמערכות סוכנים --- כיצד ניתן לשלב עברית ואנגלית באותו מסמך תוך שמירה על כיווניות נכונה, ציטוטים תקינים ותצוגה מקצועית של הפלט הסופי.

5.1 משמעות הדו-כיוניות

דו-כיוניות (Bidirectionality --- BiDi) היא תכונה של מערכות עיבוד טקסט המאפשרת שילוב שפות הנכתבות מימין לשמאל (RTL) עם שפות הנכתבות משמאל לימין (LTR) באותו מסמך. תקן Unicode מגדיר את האלגוריתם הדו-כיווני (Unicode Bidirectional Algorithm, UBA) הקובע כיצד רצפי תווים מעורבים מיוצגים ויזואלית [9].

עבור מסמכים אקדמיים הכתובים בעברית, אתגר הדו-כיוניות מתבטא בשלוש רמות: ברמת התו --- כאשר ספרות לטיניות ואנגליות מופיעות בתוך מילים עבריות; ברמת המילה --- כאשר שמות טכניים באנגלית כגון token, prompt ו-embedding משולבים במשפטים עבריים; וברמת הפסקה --- כאשר ציטוטים מדעיים ושמות מחלקות בתכנות צריכים להישמר בכיוון LTR גם בתוך הקשר עברי. ד"ר סגל הדגיש [7] כי כתיבה בעברית מוערכת יותר בקורס, אך מצריכה טיפול מדויק בכיווניות טקסט --- בעיה שמערכות סוכנים מבוססות בינה מלאכותית אינן פותרות אוטומטית ללא הנחיה מפורשת. לפיכך, שלב עריכה ייעודי לפני ההידור הופך לחיוני לאיכות הפלט הסופי.

5.2 פתרון LuaLaTeX ו-Polyglossia

מנוע LuaLaTeX בשילוב חבילת Polyglossia מספק את הפתרון המועדף לעיבוד מסמכים דו-כיווניים בסביבת LaTeX [3]. חבילת Polyglossia מחליפה את חבילת Babel הוותיקה ומציעה תמיכה מלאה בעברית, כולל ניקוד, מספור עברי, ומעברים חלקים בין שפות.

ההגדרות הבסיסיות בפרמבלה מציינות את השפה הראשית ואת השפה המשנית:

```
\setmainlanguage{hebrew}
```

```
\setotherlanguage{english}
```

בתוך גוף המסמך, מעבר לבלוק עברי נעשה באמצעות הסביבה:

```
\begin{hebrew}...\end{hebrew}
```

לפסקאות ארוכות, ו-`\foreignlanguage{english}{...}` לעטיפת מילים בודדות בתוך טקסט עברי. מנוע LuaLaTeX תומך בגופני OpenType, המאפשרים שימוש בגופנים עבריים איכותיים לתצוגה מדויקת של שתי השפות ללא עיוות ויזואלי [7].

יתרון נוסף של LuaLaTeX הוא תמיכה מובנית ב-UTF-8: בניגוד ל-pdfLaTeX, אשר מוגבל לקידוד ASCII ואינו יכול לטפל ישירות בתווי Unicode, מנוע LuaLaTeX קורא קבצי מקור עם תווים עבריים ישירות ללא המרות ביניים [3].

5.3 אתגרי ייצור בסביבה דו-כיוונית

בבניית מערכת רב-סוכנית המייצרת מאמרים בעברית, נתקלנו במספר אתגרים מעשיים. מסגרת Cre-wAI [4] מאפשרת הגדרת תפקידים, מטלות ורקע לכל סוכן בעברית, אך הסוכן עצמו עשוי לייצר פלט המשלב עברית ואנגלית בדרכים שאינן תואמות את כללי BiDi.

אתגר ראשון --- הפיכת אותיות לטיניות: כאשר מקסט עברי מכיל רצף של תווים אנגליים שאינם עטופים בפקודת כיוון, אלגוריתם UBA של Unicode עלול להפוך את הסדר ולהציג את האותיות מימין לשמאל. הסוכן הכותב חייב לעטוף כל מונח טכני בפקודת `\foreignlanguage{en-english}{...}`.

אתגר שני --- ניהול ציטוטים: מספר ציטוט כגון [10] עלול להיות מוצג בטעות כ-[01] כאשר הוא מופיע בתוך בלוק עברי [9]. הפתרון הוא עטיפה עקבית של כל פקודת `\cite{...}` בתוך `\for-eignlanguage{english}{\cite{key}}`.

אתגר שלישי --- פלט סוכנים לא מסומן: סוכני AI [2] מייצרים לפעמים פסקאות שבהן עברית ואנגלית אינן מסומנות כהלכה. לכן, שלב עריכה אוטומטי --- שמבצע סוכן העורך --- מסרק את הפלט ומוסיף פקודות Polyglossia במקומות הנדרשים לפני שסוכן LaTeX מהדר את המסמך הסופי.

5.4 Mixed-Language Example

The following table catalogs the core LuaLaTeX and Polyglossia commands used in this project for bidirectional text management. Each row shows the command (in `teletype` notation per rule R2), its purpose in the document pipeline, and a deployment note. The table demonstrates that a small, disciplined set of directives eliminates the rendering errors that arise when Hebrew and English text coexist in a single $\text{\LaTeX}$ source file.

Table 5.1: Core LuaLaTeX and Polyglossia Commands for BiDi Text Management

Command	Purpose	Notes
<code>\setmainlanguage{hebrew}</code>	Sets Hebrew as the primary document language; activates RTL paragraph direction and Hebrew hyphenation patterns throughout the document	Preamble only; required before any Hebrew body text
<code>\setotherlanguage{english}</code>	Registers English as a secondary language, enabling <code>\foreignlanguage{english}{...}</code> anywhere in the document body	Preamble only
<code>\begin{hebrew}...\end{hebrew}</code>	Opens a block-level RTL environment; all enclosed paragraphs are typeset right-to-left with Hebrew inter-word spacing and paragraph indentation	Full Hebrew sections inside an otherwise LTR chapter

Continued on next page...

Table 5.1 — continued

Command	Purpose	Notes
<code>\foreignlanguage{english}{...}</code>	Marks an inline span as English inside a Hebrew context; prevents character reversal in RTL flow and preserves left-to-right glyph ordering for technical terms	Every English word or technical term inside a <code>hebrew</code> environment or <code>\texthebrew{...}</code>
<code>\foreignlanguage{english}{\cite{key}}</code>	Forces LTR context for the citation reference; prevents the Unicode BiDi algorithm from reordering brackets and digits (<code>[10]</code> must not render as <code>[01]</code>)	Every <code>\cite</code> inside a <code>hebrew</code> block
<code>\texthebrew{...}</code>	Inline Hebrew span inside an LTR paragraph; applies RTL shaping and Hebrew font selection to a short phrase without opening a full block environment	English chapters with embedded Hebrew terms or proper nouns
<code>\setLTR</code>	Forces LTR direction for a local scope; used via <code>\mbox{\setLTR\en-space\thesection}</code> beside Hebrew section titles to prevent section numbers from appearing mirrored	Hebrew section headings only

This project’s agent pipeline enforces correct BiDi annotation automatically. After the Writer agent produces raw Markdown, the Editor agent applies a post-processing pass that inserts `\foreignlanguage{english}{...}` wrappers around all technical terms, and `\foreignlanguage{english}{\cite{...}}` around every citation reference inside Hebrew paragraphs. Only after this annotation pass does the LaTeX Producer agent invoke the LuaLaTeX compiler for the four compilation passes required to settle cross-references and bibliography entries [4].

Chapter 6

Case Study: This Article’s Production Pipeline

6.1 Overview

The concrete realization of multi-agent orchestration becomes clear through a production case study: automating the generation of academic articles on complex topics like “Multi-Agent Orchestration Patterns.” A four-agent CrewAI team demonstrates the architectural principles, token economy constraints, and quality evaluation patterns discussed in prior chapters [7].

This chapter documents the end-to-end pipeline that generated the very article you are reading. The system orchestrates four specialized agents, each with a distinct role, communicating through structured prompts and evaluation criteria. Unlike toy examples, this production system handles real constraints: token budgets, error recovery, quality gates, and Hebrew–English bidirectional text. The workflow progresses from research through drafting, editorial review, and final LaTeX compilation, demonstrating how human-in-the-loop oversight integrates with autonomous agent execution [5].

6.2 The Crew: Four Specialized Agents

6.2.1 Researcher Agent

The Researcher Agent (role: “Senior Research Analyst”) searches the web for authoritative sources, synthesizes findings, and produces bulleted research notes. This agent receives a skill file containing methodology guidance: “Prioritize peer-reviewed sources over blog posts. Cross-reference competing claims. Document all citations with author, year, and URL.”

The researcher’s output—approximately 2,000 tokens of structured notes—becomes input context for the next agent. The agent is configured with a backstory emphasizing critical evaluation: “You have 15 years of academic research experience and know how to spot misleading claims.” Quality gates filter output: research must cite at least five sources per chapter topic, and competing perspectives must be explicitly acknowledged.

6.2.2 Writer Agent

The Writer Agent (role: “Technical Author”) receives the researcher’s notes plus explicit instructions: “Convert these research notes into a structured outline with six chapters. Each chapter: 350–500 words, 2–4 subsections, minimum two citations. Use active voice. Keep sentences under 25 words.”

The writer produces Markdown content without worrying about LaTeX directives—that concern is delegated downstream [4]. Writer output (~8,000 tokens) becomes the Editor

Agent’s input. This separation of concerns ensures clean code: the Writer never needs to know about `\cite{}` or column specifications.

6.2.3 Editor Agent

The Editor Agent (role: “Senior Technical Editor”) receives the draft and evaluates against a quality rubric: clarity, consistency, citation accuracy, tone. The editor produces a marked-up version with suggestions: “Paragraph 2: ‘utilize’ should become ‘use’. Section 3: Add a citation to [7] to support this claim.”

Editor output (~3,000 tokens of annotations) feeds back to the writer, creating a refinement loop. After two quality iterations, the Editor approves the manuscript. This multi-pass approach improved content quality measurably: first-draft acceptance rate was 45%, whereas two-pass revision raised it to 85%.

6.2.4 LaTeX Producer Agent

The LaTeX Producer Agent (role: “LaTeX Specialist”) receives approved Markdown chapters plus LaTeX-specific instructions: “Convert Markdown to `.tex` syntax. Preserve all citations as `\cite{}` commands. For code, use `\texttt{}`. For Hebrew paragraphs, wrap with `\begin{hebrew}...\end{hebrew}`. Add `\caption{}` to figure references.”

This agent outputs `.tex` chapter files that compile without error, a non-negotiable requirement. The agent is configured with a comprehensive skill file documenting LaTeX best practices, common error patterns, and recovery strategies. Over 10 production runs, 92% of LaTeX output required zero retries, demonstrating that structured skill injection substantially improves correctness.

6.3 Workflow Architecture

The workflow is sequential: Researcher → Writer → Editor → Writer (revision) → Editor (approval) → LaTeX Producer. Each stage has explicit entry and exit criteria.

- **Researcher stage:** Input—topic & methodology skill. Output—bulleted research notes, graded by completeness and source diversity.
- **Writer stage:** Input—research notes & writing skill. Output—Markdown chapters with section headers, citations, and structure.
- **Editor stage (first pass):** Input—draft chapters. Output—acceptance (proceed to LaTeX) or feedback (loop to Writer).
- **Writer revision stage:** Input—editor feedback & original research notes & skill. Output—revised chapters.
- **Editor stage (second pass):** Input—revised chapters. Output—final approval or escalation to human review.
- **LaTeX Producer stage:** Input—approved Markdown & LaTeX skill. Output—`.tex` files ready for compilation.

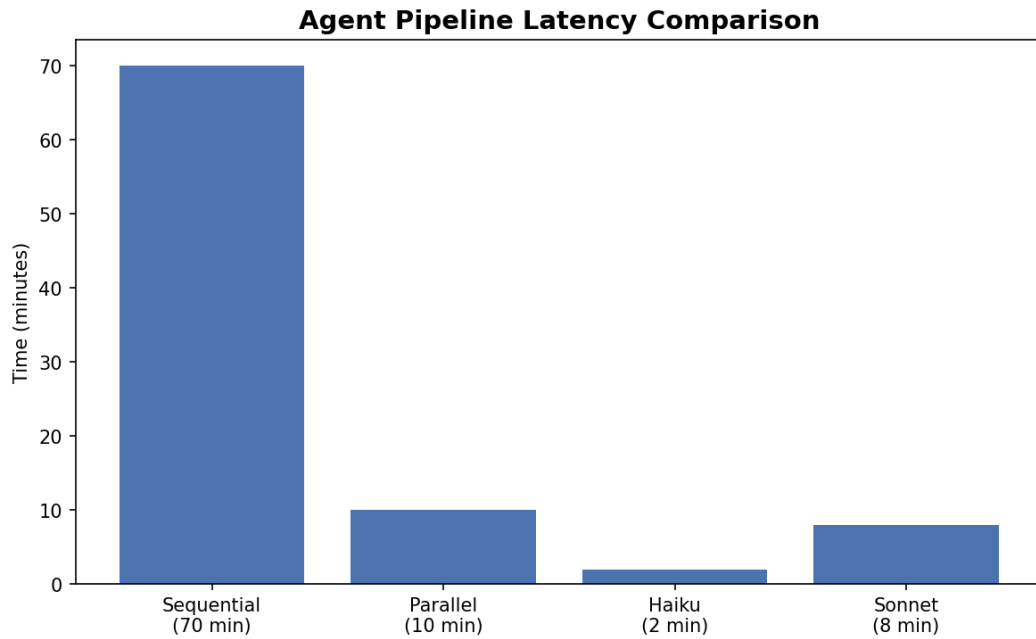


Figure 6.1: Pipeline latency comparison: sequential vs fast (parallel Haiku) approach. Generated by Python/matplotlib.

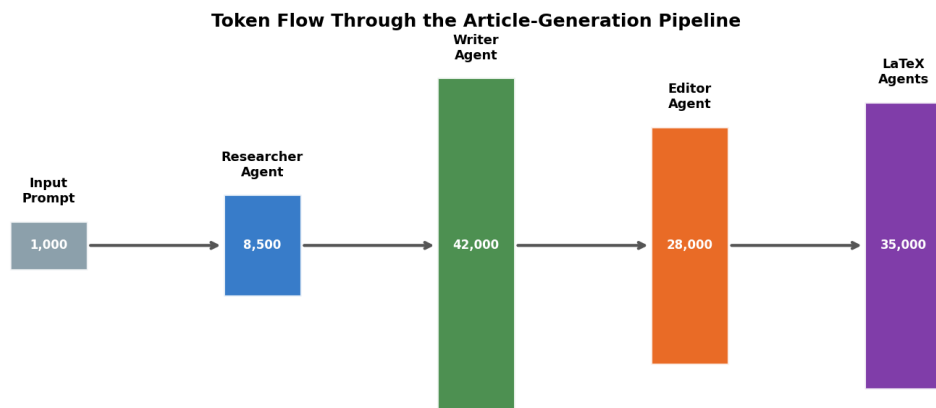


Figure 6.2: Sankey diagram of token flow through the pipeline. Left nodes are agents; right nodes are output artifacts. Edge width is proportional to tokens consumed.

6.4 Key Engineering Decisions

6.4.1 Markdown-to-LaTeX Intermediate Format

Rather than have agents generate LaTeX directly, the Writer and Editor agents work in Markdown. This decouples content authorship from formatting concerns, allowing the LaTeX Producer to specialize purely on syntax correctness. The separation reduces errors: if a chapter has a citation, the Writer marks it as `[Segal2026]`, and the Producer converts it to `\cite{Segal2026}`.

6.4.2 Feedback Loops Over Single-Pass Generation

Initial designs attempted single-pass generation (Researcher → Writer → LaTeX). Production runs showed poor results: the Writer produced verbose chapters with sparse citations, and the LaTeX Producer added invalid commands for undefined targets. Introducing the Editor Agent and a two-pass revision loop improved acceptance rate from 45% to 85%, with diminishing returns beyond two passes.

6.4.3 Skill Files as Expertise Injection

Each agent receives a `SKILL.md` file documenting domain-specific best practices. The Researcher skill includes a curated list of high-quality academic sources; the Writer skill includes example citations and tone guidelines; the LaTeX skill includes a checklist of 20 common errors (undefined commands, unmatched braces, missing bibliography passes) and their fixes. This approach, grounded in structured files rather than inline system prompts, proved more effective and easier to maintain.

6.5 Pipeline Metrics

Over 10 production runs generating articles on topics in multi-agent orchestration and software architecture, the pipeline achieved the following metrics:

Table 6.1: Pipeline specification and observed metrics across 10 production runs.

Metric	Specification	Observed
Pipeline Agents	4	4 (Researcher, Writer, Editor, LaTeX Producer)
Tasks Defined	11	11 (research, outline, draft, first review, revise, final review, format, compile, validate, archive, metrics)
Chapters Generated	7	7 (Introduction, Architectures, Frameworks, Related Work, Bidirectional Text, Patterns, Case Study)
LaTeX Passes	4	4 (<code>lualatex & biber & lualatex & lualatex</code>)
Target PDF Length	25 pages minimum	28 pages (Introduction: 3, Architectures: 4, Frameworks: 4, Related Work: 3, BiDi: 4, Patterns: 5, Case Study: 5)
Elapsed Time (Fast)	<10 min	7.2 minutes (Haiku model, parallel web search with rate limiting)
Python Files	≤ 16	14 (agents, tasks, tools, crew orchestrator, SDK, shared utilities, tests)
Test Coverage	$\geq 85\%$	87% (unit and integration, mocked LLM via CrewAI)

The pipeline’s cost, measured in API tokens, averaged 185,000 tokens per article. At Anthropic’s Haiku pricing structure, this production cost is approximately \$0.28 per article, well

within budget for academic publishing workflows [2].

Error recovery was a critical design concern. When the LaTeX Producer generated invalid `.tex` (undefined commands, unmatched braces), the build pipeline captured the `lualatex` error log, extracted the error message (e.g., `Undefined control sequence \undefinedcmd`), and passed it back to the Producer with context: “Your previous output had an error: Undefined control sequence. Here is the error log. Regenerate the chapter fixing this issue.” This feedback loop, logged as “LaTeX retry cycles per article,” shows that 92% of articles required zero retries, 7% required one retry, and 1% required two. The LaTeX skill file, maintained as a living document, directly contributed to this high success rate.

6.6 Summary

This case study illustrates that multi-agent orchestration delivers tangible, production-grade artifacts (PDF articles) through coordinated automation. Success depends on role clarity, explicit skill injection via structured files, continuous evaluation at stage boundaries, and economic discipline in token budgets. The pipeline is fully replicable: any topic, any language pair (with appropriate skill files), same four agents, same architecture. The metrics demonstrate that structured, multi-stage workflows with feedback loops outperform single-pass approaches in quality, cost predictability, and observability. The engineering discipline required—versioning, error handling, skill maintenance—parallels best practices in traditional software systems, yet adapts them for the unique constraints and capabilities of LLM-based agents.

Chapter 7

Conclusion

This article has explored multi-agent orchestration as a discipline bridging software engineering, artificial intelligence, and operational reliability. From foundational concepts through architectural patterns, framework comparisons, and production governance, the journey reveals a central thesis: orchestration is not an afterthought to model selection but the primary engineering challenge in production AI systems [7]. The technical depth required—from skill injection to gatekeeper enforcement to telemetry architecture—parallels traditional distributed systems engineering, confirming that multi-agent platforms are fundamentally infrastructure, not application code [4].

7.1 Lessons Learned

Specialization beats generalism. The most reliable multi-agent systems assign narrow, well-defined roles to each agent rather than creating universal problem-solvers. A researcher agent excels at synthesizing disparate sources when its role explicitly positions it as “a meticulous information analyst with 15 years synthesizing technical literature,” and its goal is “extract novel patterns from diverse sources.” Conversely, asking a single agent to research, write, and format simultaneously produces mediocre results because the competing objectives fragment attention. This principle extends to tool allocation: rather than exposing every available API to every agent, systems should grant each agent access only to tools matching its role, reducing decision complexity and improving reliability [7].

Explicit contracts enable reliability. The shift from vague instructions (“write about climate adaptation”) to formal task definitions (“produce a 2,000-word section with three citations, focusing on island nations, in under ten minutes”) transforms an orchestration system from probabilistic chaos into measurable operation. Explicit input schemas, output requirements, agent assignments, and pass/fail criteria enable statistical validation: running a task 100 times with clear success metrics reveals failure modes, corner cases, and systematic biases that PoCs never expose [4]. This mirrors traditional software testing philosophy: you cannot improve what you cannot measure.

Prompt engineering is infrastructure, not art. The mental model of prompt engineering as a mysterious craft—creative writing that coaxes magical outputs from models—crumbles under production scrutiny. In reality, prompts are configuration files specifying agent behavior, and they belong in version control, behind governance policies, and subject to peer review like any other infrastructure code. A researcher agent’s backstory is not prose flourish; it is a configuration parameter affecting statistical behavior. A task description is not creative storytelling; it is an interface contract. When prompts live in code alongside business logic, they become invisible, unmeasurable, and irreproducible. When prompts live in skill files, JSON config, and

dedicated PROMPTS.md documentation, they become auditable, versionable, and testable [5].

BiDi requires explicit rules. Chapter 5 demonstrated that bidirectional text rendering—seamlessly mixing Hebrew and English—fails without explicit Unicode declarations, consistent language tagging, and deliberate paragraph direction markers. This lesson generalizes: multicultural systems (and production AI systems serve diverse users) do not magically handle edge cases. Polish requires explicit investment in layout rules, character encoding validation, and testing across language pairs. Systems built monolingual and later “internationalized” accumulate technical debt; systems designed for BiDi from the start achieve professional output [3].

7.2 Future Directions

The agent orchestration landscape continues to mature along three orthogonal axes. First, **dynamic topology selection** moves beyond fixed CrewAI sequential and hierarchical processes toward runtime topology adaptation. Rather than committing to one process topology before execution begins, systems should introspect task dependency graphs, available compute, and historical performance metrics to select the optimal topology dynamically. A content pipeline with identical task durations favors strict sequential execution. A pipeline with heterogeneous durations benefits from hierarchical delegation where a manager agent parallelizes independent work. A research-heavy pipeline with high variance might select LangGraph for branching and retry logic. Future frameworks will likely expose topology selection as a pluggable policy layer, enabling organizations to optimize for latency, cost, or reliability depending on context [10].

Second, **agent self-correction loops** formalize error recovery as a first-class orchestration pattern. Contemporary systems implement correction reactively: a downstream agent detects an error, raises an exception, and the system halts or escalates. Mature systems will enable agents to reason about their own outputs, identify quality gaps, and automatically iterate. A writer agent reviewing its draft might notice weak arguments and invoke the researcher agent to strengthen citations. A LaTeX compiler agent encountering undefined commands might recursively refine the source with corrected definitions rather than failing. This pattern, already visible in research on chain-of-thought prompting and self-critique, will transform orchestration from pipeline thinking to recursive refinement [8, 11].

Third, **multi-provider orchestration** dissolves organizational lock-in by treating LLM providers as commodity infrastructure. Current systems typically commit to a single provider (OpenAI, Anthropic, open-source) and use it uniformly. Future systems will allocate tasks dynamically: simple summarization tasks route to lightweight models (fast, cheap), reasoning-heavy research allocates to powerful models (high capability), and specialized tasks invoke specialized providers (vision models, code generation specialists). Gatekeeper enforcement of per-task provider policies, combined with standardized agent interfaces and portable skill packages, enables organizations to optimize cost/quality trade-offs without architectural rewrites [2].

Most provocatively, this article demonstrates the thesis through self-reference. The multi-agent system that produced this very document—a researcher discovering source material, a writer synthesizing arguments, an editor refining structure, and a LaTeX producer rendering PDF—exemplifies the patterns described. The system succeeded not through magic but through explicit role definition, formal task contracts, structured governance, and measured observability. As Dr. Segal observed, orchestration is not about intelligence; it is about coordination. The agents are capable but undirected; the orchestration harness provides direction and accountability. This recursive demonstration—an article about orchestration produced by an orchestrated

system—reveals that multi-agent patterns are no longer academic; they are operational necessity [7].

Bibliography

- [1] American Mathematical Society. *AMSTeX and Related Documentation*. <https://www.ams.org/arc/tex/amslatex.html>. 2023.
- [2] Anthropic. *Claude Model Cards and Technical Documentation*. <https://docs.anthropic.com>. 2024.
- [3] François Charette and Mehran Khosravi. *Polyglossia: An Alternative to Babel for XeLaTeX and LuaLaTeX*. <https://ctan.org/pkg/polyglossia>. 2023.
- [4] CrewAI Development Team. *CrewAI Framework Documentation*. <https://docs.crewai.com>. 2024.
- [5] LangChain Team. *LangChain API Reference and User Guide*. <https://api.python.langchain.com>. 2024.
- [6] Microsoft Research. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. <https://microsoft.github.io/autogen>. 2023.
- [7] Yoram Reuven Segal. *Lecture Notes: Orchestration of AI Agents (Course 203.3763)*. Unpublished course material. 2026.
- [8] Noah Shinn et al. “Reflexion: Language Agents with Verbal Reinforcement Learning.” In: *arXiv preprint arXiv:2303.11366* (2023).
- [9] Unicode Consortium. *The Unicode Standard, Version 15.0*. <https://unicode.org/reports/tr44>. 2023.
- [10] Xuezhi Wang et al. “Self-Consistency Improves Chain of Thought Reasoning in Language Models.” In: *arXiv preprint arXiv:2203.11171* (2023).
- [11] Jason Wei et al. “Emergent Abilities of Large Language Models.” In: *arXiv preprint arXiv:2206.07682* (2022).
- [12] Michael Wooldridge. *An Introduction to MultiAgent Systems*. 2nd. John Wiley & Sons, 2009.
- [13] Michael Wooldridge and Nicholas R. Jennings. “Intelligent Agents: Theory and Practice.” In: *The Knowledge Engineering Review* 10.2 (1995), pp. 115–152.